PYTHON ACTIVITY-BASED LEARNING
10 EASY CASE STUDIES

Assignment documentation with algorithms, flowcharts, pseudocode, Python programs, test cases, and sample outputs

Name: _____ Roll No.: _____

Course/Section: _____ Date: _____

Introduction

This assignment presents ten beginner-friendly Python case studies selected from the Easy category in the supplied PDF. The examples demonstrate input/output, arithmetic, conditionals, loops, and validation.

General Requirements

- Python 3 installed
- Text editor or IDE such as IDLE, VS Code, or PyCharm
- Save each program in a separate .py file
- Run and verify with the test cases

Case Study 2: Progressive Electricity Billing

1. Aim

Calculate an electricity bill using progressive unit slabs.

2. Algorithm

1. Read units consumed.
2. Charge the first 100 units at ■1.50 per unit.
3. Charge units 101–200 at ■2.50 per unit.
4. Charge units above 200 at ■4.00 per unit.
5. Display the slab-wise charges and total bill.

3. Flowchart

START
↓
Input values
↓
Validate input
↓
[Invalid?] Yes → Show error → STOP
No ↓
Calculate / apply conditions
↓
Display result
↓
STOP

4. Pseudocode

```
START
INPUT units
IF units < 0: PRINT invalid input
ELSE
  IF units <= 100: bill = units × 1.50
  ELSE IF units <= 200: bill = 100 × 1.50 + (units - 100) × 2.50
  ELSE: bill = 100 × 1.50 + 100 × 2.50 + (units - 200) × 4.00
  PRINT bill
STOP
```

5. Python Program

```
units = float(input("Enter units consumed: "))

if units < 0:
    print("Invalid units. Enter a non-negative value.")
elif units <= 100:
    bill = units * 1.50
    print(f"Electricity bill: ■{bill:.2f}")
elif units <= 200:
    bill = 100 * 1.50 + (units - 100) * 2.50
    print(f"Electricity bill: ■{bill:.2f}")
else:
    bill = 100 * 1.50 + 100 * 2.50 + (units - 200) * 4.00
    print(f"Electricity bill: ■{bill:.2f}")
```

6. Test Cases

Input / Scenario	Expected result
80	■120.00
150	■275.00
250	■600.00
-5	Invalid units

7. Sample Output

Example test: 80
Expected result: ■120.00

8. Documentation / Conclusion

The rates are example rates for this assignment, not an official utility tariff.
Conclusion: This program demonstrates Python input, variables, arithmetic, conditions, loops where applicable, and validation.

Case Study 3: Hospital Billing and Patient Classification

1. Aim

Calculate a simple hospital bill and classify the patient by total charges.

2. Algorithm

1. Read consultation fee, number of bed-days, and daily bed charge.
2. Calculate total = consultation fee + bed-days × daily bed charge.
3. Classify total below ■5,000 as Basic, ■5,000–■15,000 as Standard, and above ■15,000 as Premium.
4. Reject negative inputs and print the result.

3. Flowchart

START
↓
Input values
↓
Validate input
↓
[Invalid?] Yes → Show error → STOP
No ↓
Calculate / apply conditions
↓
Display result
↓
STOP

4. Pseudocode

```
START
INPUT consultation_fee, bed_days, daily_charge
IF any input is negative: PRINT invalid input
ELSE total = consultation_fee + bed_days × daily_charge
IF total < 5000: category = Basic
ELSE IF total <= 15000: category = Standard
ELSE category = Premium
PRINT total and category
STOP
```

5. Python Program

```
consultation_fee = float(input("Consultation fee (■): "))
bed_days = int(input("Number of bed-days: "))
daily_charge = float(input("Daily bed charge (■): "))

if consultation_fee < 0 or bed_days < 0 or daily_charge < 0:
    print("Invalid input. Values cannot be negative.")
else:
    total = consultation_fee + bed_days * daily_charge
    if total < 5000:
        category = "Basic"
    elif total <= 15000:
        category = "Standard"
    else:
        category = "Premium"

    print(f"Total hospital bill: ■{total:.2f}")
    print(f"Patient billing category: {category}")
```

6. Test Cases

Input / Scenario	Expected result
1000, 2 days, ■1000/day	■3000; Basic
2000, 5 days, ■1500/day	■9500; Standard
3000, 10 days, ■1500/day	■18000; Premium

7. Sample Output

Example test: 1000, 2 days, ■1000/day
Expected result: ■3000; Basic

8. Documentation / Conclusion

This is a billing-category demonstration only; it does not classify medical urgency or patient health.
Conclusion: This program demonstrates Python input, variables, arithmetic, conditions, loops where applicable, and validation.

Case Study 4: Employee Payroll and Deduction Analysis

1. Aim

Compute an employee's net salary after simple percentage deductions.

2. Algorithm

1. Read employee name, basic salary, and deduction percentage.
2. Validate that salary is non-negative and deduction percentage is between 0 and 100.
3. Calculate deduction = salary × percentage / 100.
4. Calculate net salary = salary – deduction.
5. Display gross salary, deduction, and net salary.

3. Flowchart

START
↓
Input values
↓
Validate input
↓
[Invalid?] Yes → Show error → STOP
No ↓
Calculate / apply conditions
↓
Display result
↓
STOP

4. Pseudocode

```
START
INPUT name, salary, deduction_percent
IF salary < 0 OR percent < 0 OR percent > 100: PRINT invalid input
ELSE deduction = salary × percent / 100
net_salary = salary - deduction
PRINT name, salary, deduction, net_salary
STOP
```

5. Python Program

```
name = input("Employee name: ")
salary = float(input("Basic salary (■): "))
deduction_percent = float(input("Deduction percentage (0–100): "))

if salary < 0 or not 0 <= deduction_percent <= 100:
    print("Invalid salary or deduction percentage.")
else:
    deduction = salary * deduction_percent / 100
    net_salary = salary - deduction
    print(f"Employee: {name}")
    print(f"Gross salary: ■{salary:.2f}")
    print(f"Deduction: ■{deduction:.2f}")
    print(f"Net salary: ■{net_salary:.2f}")
```

6. Test Cases

Input / Scenario	Expected result
Ravi, ■20000, 10%	Deduction ■2000; net ■18000
Maya, ■35000, 5%	Deduction ■1750; net ■33250
Sam, ■12000, 100%	Net ■0

7. Sample Output

Example test: Ravi, ■20000, 10%
Expected result: Deduction ■2000; net ■18000

8. Documentation / Conclusion

The deduction percentage is entered by the user; it is not a statement of statutory tax or payroll rules.
Conclusion: This program demonstrates Python input, variables, arithmetic, conditions, loops where applicable, and validation.

Case Study 7: Retail Billing with Multiple Discount Rules

1. Aim

Calculate a retail bill with a discount based on purchase amount.

2. Algorithm

1. Read the purchase amount.
2. Reject a negative amount.
3. Apply 10% discount for amounts of ■5,000 or more; otherwise apply 5% discount for amounts of ■2,000 or more; otherwise no discount.
4. Calculate final amount and display discount and payable total.

3. Flowchart

START
↓
Input values
↓
Validate input
↓
[Invalid?] Yes → Show error → STOP
No ↓
Calculate / apply conditions
↓
Display result
↓
STOP

4. Pseudocode

```
START
INPUT amount
IF amount < 0: PRINT invalid input
ELSE IF amount >= 5000: rate = 10
ELSE IF amount >= 2000: rate = 5
ELSE rate = 0
discount = amount × rate / 100
payable = amount - discount
PRINT rate, discount, payable
STOP
```

5. Python Program

```
amount = float(input("Enter purchase amount (■): "))

if amount < 0:
    print("Invalid amount.")
else:
    if amount >= 5000:
        rate = 10
    elif amount >= 2000:
        rate = 5
    else:
        rate = 0

    discount = amount * rate / 100
    payable = amount - discount
    print(f"Discount rate: {rate}%")
    print(f"Discount amount: ■{discount:.2f}")
    print(f"Final amount: ■{payable:.2f}")
```

6. Test Cases

Input / Scenario	Expected result
■1500	0%; payable ■1500
■3000	5%; discount ■150; payable ■2850
■6000	10%; discount ■600; payable ■5400

7. Sample Output

Example test: ■1500
Expected result: 0%; payable ■1500

8. Documentation / Conclusion

These discount thresholds are example rules for the assignment.
Conclusion: This program demonstrates Python input, variables, arithmetic, conditions, loops where applicable, and validation.

Case Study 8: Manufacturing Cost and Profitability Analysis

1. Aim

Calculate production cost, revenue, profit or loss, and profit margin.

2. Algorithm

1. Read quantity produced, cost per item, and selling price per item.
2. Validate that all numeric values are non-negative.
3. Calculate total cost = quantity × cost per item.
4. Calculate revenue = quantity × selling price.
5. Calculate profit = revenue – total cost; if revenue is positive, margin = profit / revenue × 100.
6. Display results and indicate profit, loss, or break-even.

3. Flowchart

START
↓
Input values
↓
Validate input
↓
[Invalid?] Yes → Show error → STOP
No ↓
Calculate / apply conditions
↓
Display result
↓
STOP

4. Pseudocode

```
START
INPUT quantity, cost_per_item, selling_price
IF any value is negative: PRINT invalid input
ELSE cost = quantity × cost_per_item
revenue = quantity × selling_price
profit = revenue - cost
IF revenue > 0: margin = profit / revenue × 100 ELSE margin = 0
IF profit > 0: status = Profit; ELSE IF profit < 0: status = Loss; ELSE status = Break-even
PRINT results
STOP
```

5. Python Program

```
quantity = int(input("Quantity produced: "))
cost_per_item = float(input("Cost per item (■): "))
selling_price = float(input("Selling price per item (■): "))

if quantity < 0 or cost_per_item < 0 or selling_price < 0:
    print("Invalid input. Values cannot be negative.")
else:
    total_cost = quantity * cost_per_item
    revenue = quantity * selling_price
    profit = revenue - total_cost
    margin = (profit / revenue * 100) if revenue > 0 else 0

    if profit > 0:
        status = "Profit"
    elif profit < 0:
        status = "Loss"
    else:
        status = "Break-even"

    print(f"Total production cost: ■{total_cost:.2f}")
    print(f"Total revenue: ■{revenue:.2f}")
    print(f"Profit/Loss: ■{profit:.2f}")
    print(f"Status: {status}")
    print(f"Profit margin: {margin:.2f}%")
```

6. Test Cases

Input / Scenario	Expected result
100 items, cost ■20, selling ■30	Cost ■2000; revenue ■3000; profit ■1000; margin 33.33%
50, ■40, ■30	Loss ■500; margin -33.33%

Input / Scenario	Expected result
10, ■25, ■25	Break-even ■0

7. Sample Output

Example test: 100 items, cost ■20, selling ■30
Expected result: Cost ■2000; revenue ■3000; profit ■1000; margin 33.33%

8. Documentation / Conclusion

Profit margin is calculated as profit divided by revenue; it is 0 when revenue is zero.
Conclusion: This program demonstrates Python input, variables, arithmetic, conditions, loops where applicable, and validation.

Case Study 9: Multi-Currency Travel Budget Analysis

1. Aim

Convert travel expenses from a foreign currency to INR and compare them with a budget.

2. Algorithm

1. Read the travel budget in INR, foreign-currency expenses, and exchange rate (INR per foreign unit).
2. Validate that values are non-negative and exchange rate is greater than zero.
3. Convert expenses to INR.
4. Calculate remaining budget = budget – converted expenses.
5. Report whether the traveler is within budget or over budget.

3. Flowchart

START
↓
Input values
↓
Validate input
↓
[Invalid?] Yes → Show error → STOP
No ↓
Calculate / apply conditions
↓
Display result
↓
STOP

4. Pseudocode

```
START
INPUT budget_inr, foreign_expenses, exchange_rate
IF budget < 0 OR expenses < 0 OR rate <= 0: PRINT invalid input
ELSE expenses_inr = foreign_expenses × exchange_rate
remaining = budget_inr - expenses_inr
IF remaining >= 0: PRINT within budget and remaining
ELSE PRINT over budget and excess
STOP
```

5. Python Program

```
budget_inr = float(input("Travel budget in INR (■): "))
foreign_expenses = float(input("Expenses in foreign currency: "))
exchange_rate = float(input("INR per 1 foreign-currency unit: "))

if budget_inr < 0 or foreign_expenses < 0 or exchange_rate <= 0:
    print("Invalid input.")
else:
    expenses_inr = foreign_expenses * exchange_rate
    remaining = budget_inr - expenses_inr
    print(f"Expenses in INR: ■{expenses_inr:.2f}")
    if remaining >= 0:
        print(f"Within budget. Balance: ■{remaining:.2f}")
    else:
        print(f"Over budget by: ■{-remaining:.2f}")
```

6. Test Cases

Input / Scenario	Expected result
Budget ■20000, expenses 100, rate 80	Expenses ■8000; balance ■12000
Budget ■10000, expenses 200, rate 75	Expenses ■15000; over by ■5000

7. Sample Output

Example test: Budget ■20000, expenses 100, rate 80
Expected result: Expenses ■8000; balance ■12000

8. Documentation / Conclusion

Enter the exchange rate yourself. Rates are illustrative and are not live market rates.
Conclusion: This program demonstrates Python input, variables, arithmetic, conditions, loops where applicable, and validation.

Case Study 10: Campus Event Financial Planning

1. Aim

Estimate event costs, total funds available, and surplus or shortfall.

2. Algorithm

1. Read expected attendees, cost per attendee, fixed cost, and sponsorship amount.
2. Reject negative values.
3. Calculate variable cost = attendees × cost per attendee.
4. Calculate total cost = variable cost + fixed cost.
5. Calculate balance = sponsorship – total cost.
6. Report surplus or additional funds required.

3. Flowchart

START
↓
Input values
↓
Validate input
↓
[Invalid?] Yes → Show error → STOP
No ↓
Calculate / apply conditions
↓
Display result
↓
STOP

4. Pseudocode

```
START
INPUT attendees, cost_per_attendee, fixed_cost, sponsorship
IF any value is negative: PRINT invalid input
ELSE variable_cost = attendees × cost_per_attendee
total_cost = variable_cost + fixed_cost
balance = sponsorship - total_cost
IF balance >= 0: PRINT surplus
ELSE PRINT shortfall
STOP
```

5. Python Program

```
attendees = int(input("Expected attendees: "))
cost_per_attendee = float(input("Cost per attendee (■): "))
fixed_cost = float(input("Fixed event cost (■): "))
sponsorship = float(input("Sponsorship/funds available (■): "))

if attendees < 0 or cost_per_attendee < 0 or fixed_cost < 0 or sponsorship < 0:
    print("Invalid input. Values cannot be negative.")
else:
    variable_cost = attendees * cost_per_attendee
    total_cost = variable_cost + fixed_cost
    balance = sponsorship - total_cost
    print(f"Variable cost: ■{variable_cost:.2f}")
    print(f"Total event cost: ■{total_cost:.2f}")
    if balance >= 0:
        print(f"Surplus funds: ■{balance:.2f}")
    else:
        print(f"Additional funds required: ■{-balance:.2f}")
```

6. Test Cases

Input / Scenario	Expected result
100 attendees, ■200 each, fixed ■5000, funds ■30000	Cost ■25000; surplus ■5000
200, ■250, ■10000, funds ■40000	Cost ■60000; shortfall ■20000

7. Sample Output

Example test: 100 attendees, ■200 each, fixed ■5000, funds ■30000
Expected result: Cost ■25000; surplus ■5000

8. Documentation / Conclusion

This is a planning estimate and does not include unentered costs.
Conclusion: This program demonstrates Python input, variables, arithmetic, conditions, loops where applicable, and validation.

Case Study 11: Examination Result and Grace-Mark Validation

1. Aim

Calculate total and percentage, and determine pass/fail using a simple example rule.

2. Algorithm

1. Read marks for three subjects and the maximum mark per subject.
2. Validate marks are within 0 and maximum marks.
3. Calculate total and percentage.
4. For this example, a student passes if each subject mark is at least 35.
5. Display total, percentage, and result.

3. Flowchart

START
↓
Input values
↓
Validate input
↓
[Invalid?] Yes → Show error → STOP
No ↓
Calculate / apply conditions
↓
Display result
↓
STOP

4. Pseudocode

```
START
INPUT marks1, marks2, marks3, maximum
IF maximum <= 0 OR any mark < 0 OR any mark > maximum: PRINT invalid marks
ELSE total = marks1 + marks2 + marks3
percentage = total / (3 × maximum) × 100
IF all marks >= 35: result = PASS ELSE result = FAIL
PRINT total, percentage, result
STOP
```

5. Python Program

```
maximum = float(input("Maximum marks per subject: "))
marks1 = float(input("Marks in subject 1: "))
marks2 = float(input("Marks in subject 2: "))
marks3 = float(input("Marks in subject 3: "))

marks = [marks1, marks2, marks3]
if maximum <= 0 or any(mark < 0 or mark > maximum for mark in marks):
    print("Invalid marks or maximum marks.")
else:
    total = sum(marks)
    percentage = total / (3 * maximum) * 100
    result = "PASS" if all(mark >= 35 for mark in marks) else "FAIL"
    print(f"Total marks: {total:g} / {3 * maximum:g}")
    print(f"Percentage: {percentage:.2f}%")
    print(f"Result: {result}")
```

6. Test Cases

Input / Scenario	Expected result
Maximum 100; marks 70, 65, 80	Total 215/300; 71.67%; PASS
Maximum 100; marks 80, 30, 90	Total 200/300; 66.67%; FAIL
Maximum 100; marks 35, 35, 35	PASS

7. Sample Output

Example test: Maximum 100; marks 70, 65, 80
Expected result: Total 215/300; 71.67%; PASS

8. Documentation / Conclusion

The 35-mark pass rule is a simplified example. Grace marks are not added in this version despite the case-study title.
Conclusion: This program demonstrates Python input, variables, arithmetic, conditions, loops where applicable, and validation.

Case Study 12: Attendance Recovery Computation

1. Aim

Calculate attendance percentage and determine whether the minimum attendance requirement is met.

2. Algorithm

1. Read classes held, classes attended, and required percentage.
2. Validate counts and percentage range.
3. Calculate attendance percentage when classes held is greater than zero.
4. Compare it with the required percentage and report eligibility.
5. If attendance is low, calculate the minimum additional consecutive classes to attend to reach the threshold, assuming no more classes are missed.

3. Flowchart

START
↓
Input values
↓
Validate input
↓
[Invalid?] Yes → Show error → STOP
No ↓
Calculate / apply conditions
↓
Display result
↓
STOP

4. Pseudocode

```
START
INPUT held, attended, required_percent
IF held < 0 OR attended < 0 OR attended > held OR required outside 0..100: PRINT invalid input
ELSE IF held = 0: PRINT attendance unavailable
ELSE percentage = attended / held × 100
IF percentage >= required: PRINT eligible
ELSE find smallest x >= 0 where (attended + x)/(held + x) × 100 >= required
PRINT percentage and x
STOP
```

5. Python Program

```
held = int(input("Total classes held: "))
attended = int(input("Classes attended: "))
required = float(input("Required attendance percentage: "))

if held < 0 or attended < 0 or attended > held or not 0 <= required <= 100:
    print("Invalid attendance details.")
elif held == 0:
    print("Attendance percentage cannot be calculated because no classes were held.")
else:
    percentage = attended / held * 100
    print(f"Attendance: {percentage:.2f}%")
    if percentage >= required:
        print("Attendance requirement met.")
    else:
        print("Attendance requirement not yet met.")
        if required == 100:
            extra = held - attended
        else:
            extra = max(0, int((required * held - 100 * attended) // (100 - required)) + 1)
            while (attended + extra) / (held + extra) * 100 < required:
                extra += 1
        print(f"Attend the next {extra} consecutive classes without absence to reach the requirement.")
```

6. Test Cases

Input / Scenario	Expected result
Held 40, attended 32, required 75	80%; requirement met
Held 40, attended 25, required 75	62.5%; need 15 consecutive classes
Held 0, attended 0, required 75	Cannot calculate

7. Sample Output

Example test: Held 40, attended 32, required 75
Expected result: 80%; requirement met

8. Documentation / Conclusion

The recovery estimate assumes every future class is attended and uses the supplied threshold.
Conclusion: This program demonstrates Python input, variables, arithmetic, conditions, loops where applicable, and validation.

Case Study 13: ATM Transaction Sequence Simulation

1. Aim

Simulate a sequence of deposits and withdrawals while checking available balance.

2. Algorithm

1. Read starting balance and number of transactions.
2. For each transaction, read type and amount.
3. Accept deposits; accept withdrawals only when sufficient funds exist.
4. Reject negative amounts and unknown transaction types.
5. Display the balance after each transaction and the final balance.

3. Flowchart

START
↓
Input values
↓
Validate input
↓
[Invalid?] Yes → Show error → STOP
No ↓
Calculate / apply conditions
↓
Display result
↓
STOP

4. Pseudocode

```
START
INPUT starting_balance, transaction_count
IF starting_balance < 0 OR transaction_count < 0: PRINT invalid input
ELSE balance = starting_balance
REPEAT transaction_count times
  INPUT type, amount
  IF amount < 0: PRINT invalid amount
  ELSE IF type = deposit: balance = balance + amount
  ELSE IF type = withdraw AND amount <= balance: balance = balance - amount
  ELSE IF type = withdraw: PRINT insufficient funds
  ELSE PRINT unknown transaction
  PRINT balance
PRINT final balance
STOP
```

5. Python Program

```
balance = float(input("Starting balance (■): "))
count = int(input("Number of transactions: "))

if balance < 0 or count < 0:
    print("Invalid starting balance or transaction count.")
else:
    for i in range(count):
        transaction_type = input(f"Transaction {i + 1} (deposit/withdraw): ").strip().lower()
        amount = float(input("Amount (■): "))

        if amount < 0:
            print("Invalid amount.")
        elif transaction_type == "deposit":
            balance += amount
            print("Deposit successful.")
        elif transaction_type == "withdraw":
            if amount <= balance:
                balance -= amount
                print("Withdrawal successful.")
            else:
                print("Insufficient funds.")
        else:
            print("Unknown transaction type.")

        print(f"Current balance: ■{balance:.2f}")

    print(f"Final balance: ■{balance:.2f}")
```

6. Test Cases

Input / Scenario	Expected result
Starting ■1000; deposit ■500; withdraw ■200	Final balance ■1300
Starting ■300; withdraw ■500	Insufficient funds; balance ■300
Starting ■100; deposit ■0	Balance ■100

7. Sample Output

Example test: Starting ■1000; deposit ■500; withdraw ■200
Expected result: Final balance ■1300

8. Documentation / Conclusion

This is a classroom simulation only, not connected to a real bank or ATM.
Conclusion: This program demonstrates Python input, variables, arithmetic, conditions, loops where applicable, and validation.

Overall Conclusion

These ten case studies provide practice in converting real-world requirements into algorithms and Python programs. Testing normal, boundary, and invalid inputs helps verify program behavior.